SIMON SAYS · ESP32 · BOARD 1 OF 4

Simon Says, in blocks

The same memory game the ESP32 runs, rewritten as puzzle blocks. Each block does one small job. Read a stack from top to bottom, like a recipe. When a block wraps around other blocks, it repeats them or decides whether they run.

HOW THE GAME PLAYS

1. The lights flash a pattern, one LED at a time.
2. You press the buttons in the same order. Button 1 goes with LED 1, and so on.
3. Get it right and the pattern grows by one step, and plays a little faster.
4. Press the wrong button, or wait 5 seconds without pressing, and you get a long buzz: game over. Reset ends the game at any time.

THE PARTS

- **4 LEDs (lights)**
  pins 27 · 32 · 14 · 33
- **4 game buttons**
  pins A4 · A3 · A2 · A1
- **Reset button**
  pin 15
- **Buzzer**
  pin A5 · 2000 Hz tone
- **Computer screen**
  "say" blocks print messages here over USB

WHAT THE BLOCK COLOURS MEAN

- **Events** start when something happens
- **Control** repeat, wait, decide
- **Lights** LEDs and messages
- **Sound** the buzzer
- **Buttons** is a button held down?
- **Maths** compare, count, pick random
- **Memory** numbers the game keeps
- **Pattern list** the steps to repeat
- **My blocks**: a shortcut name for a whole stack of blocks, built on another board

SCRIPT A · POWER ON



SCRIPT B · THE MAIN LOOP



WHAT'S GOING ON

1. Script A runs once, the moment the board gets power.
2. Every part gets set up: buttons so they can be read, LEDs so they can be switched, the buzzer tuned to its note.
3. A short test beep, so you can hear the buzzer is wired up right.
4. The chip makes its own random numbers, so the pattern is different every game.
5. Once setup is done, this loop runs over and over until the power is pulled.
6. Pink blocks are shortcuts. "play game" stands for the whole stack on board 2, and "wait for start" is on board 4.
7. A short "ready again" beep after every game, before the lights start circling.

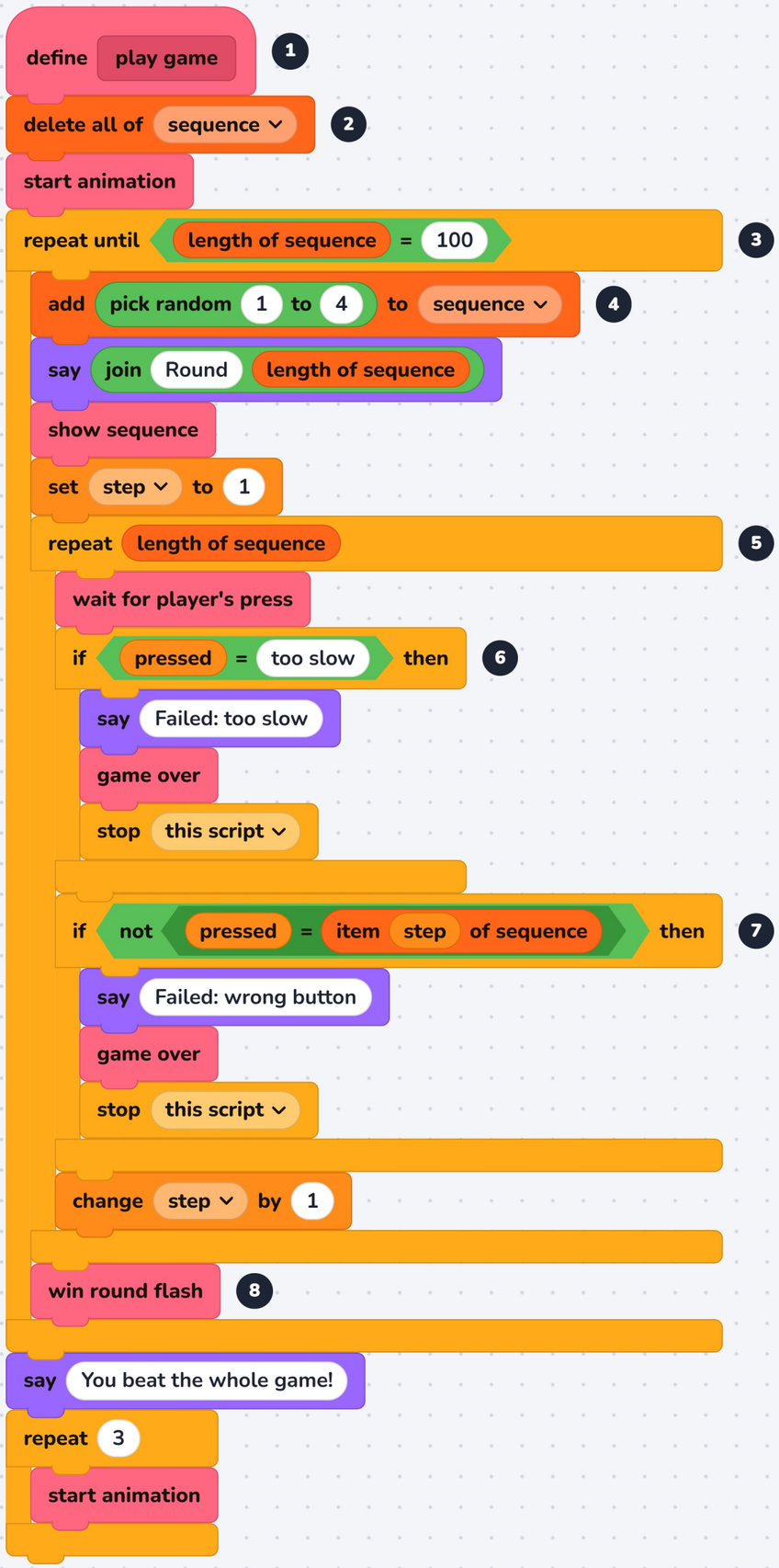
Next: the game itself →

SIMON SAYS · ESP32 · BOARD 2 OF 4

The game itself

This is what the pink "play game" block on board 1 does. Each trip around the big orange loop is one round: add a step, show the pattern, check the player's answer.

SCRIPT C · PLAY GAME



SCRIPT D · THE RESET BUTTON



WHAT THE GAME REMEMBERS

sequence	
1	3
2	1
3	4
4	4
5	2
length 5	

An example after five rounds: light LED 3, then 1, then 4, 4, 2. **step** marks which one the player is on, and **pressed** holds the button they just pushed.

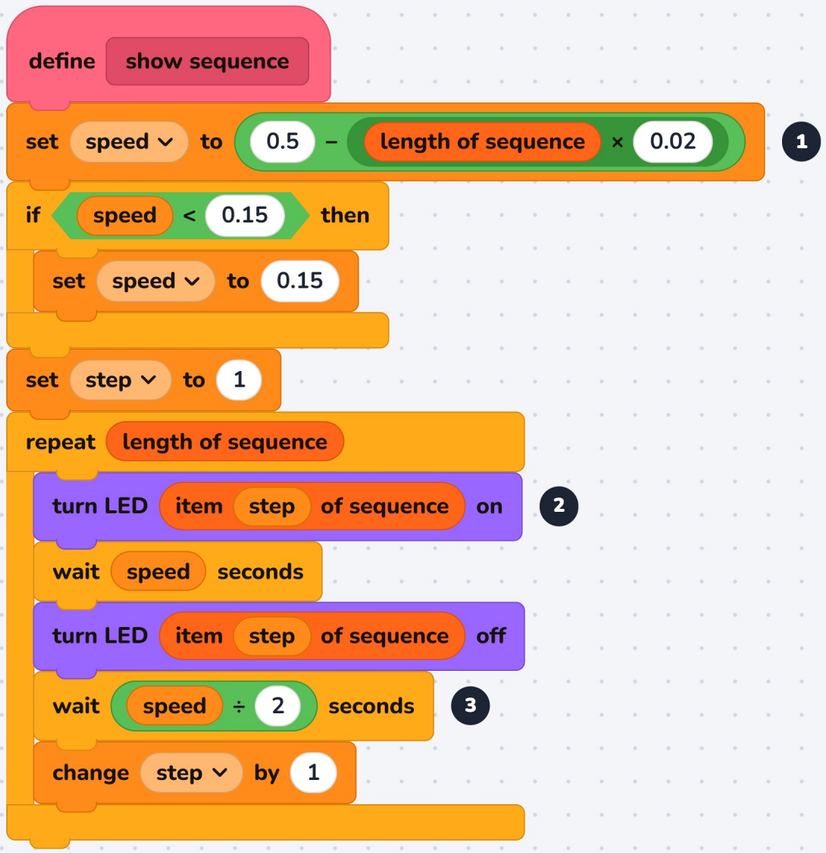
WHAT'S GOING ON

1. "define" makes a new block. Everything stacked under this hat runs whenever "play game" is used.
2. Forget the last game's pattern and start empty.
3. One trip round this loop is one round. The pattern can grow to 100 steps.
4. Add one new random light, 1 to 4, to the end of the pattern.
5. The player's turn: one press for every step of the pattern, checked one at a time.
6. No press within 5 seconds ends the game.
7. The press has to match this step of the pattern. Any other button ends the game.
8. All the lights blink once: round cleared, on to a longer pattern.
9. Reset is checked all the time while the pattern plays and while the game waits for a press. It ends the game straight away and goes back to the start.

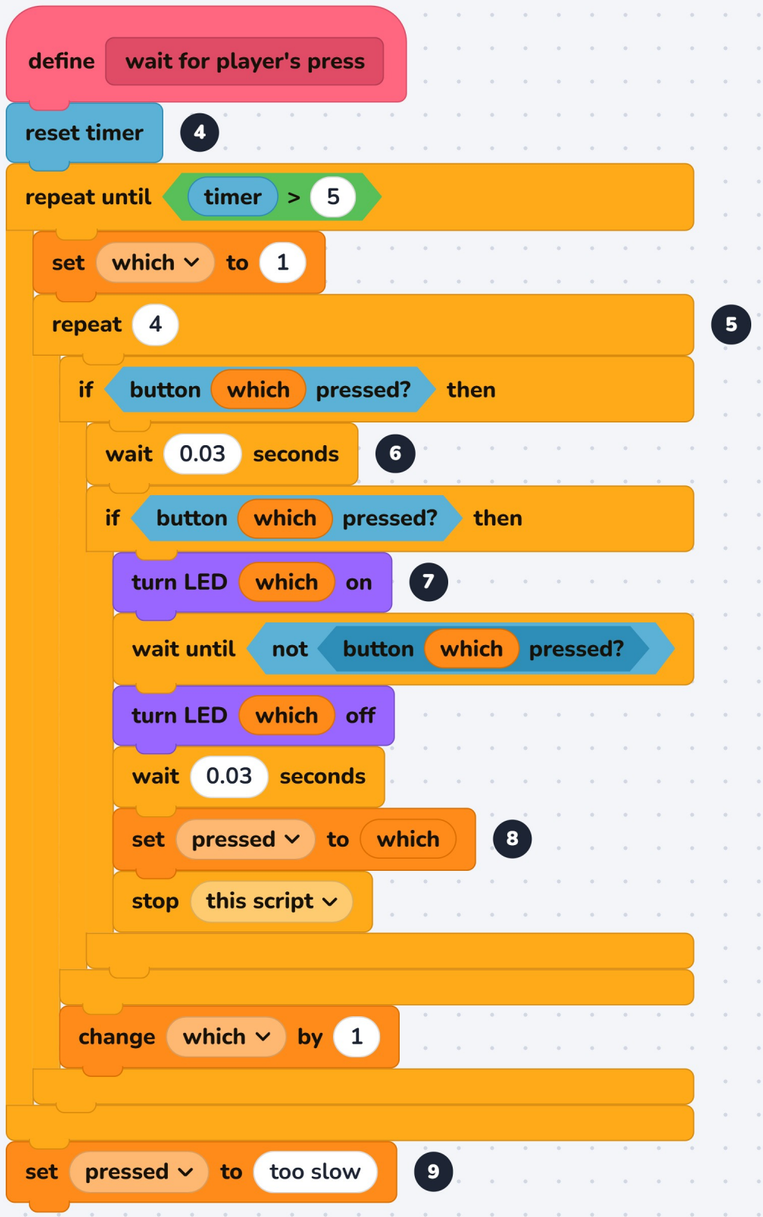
Showing the pattern, listening for presses

Two of the pink shortcut blocks used on board 2. One plays the pattern on the LEDs. The other waits for the player and works out which button they pressed.

SCRIPT E · SHOW SEQUENCE



SCRIPT F · WAIT FOR PLAYER'S PRESS



WHAT'S GOING ON

1. In round 1 each light stays on for half a second. Every round takes 0.02 s off that, but it never gets faster than 0.15 s.
2. Light the LED for this step of the pattern, then move on to the next step.
3. The gap between lights is half as long as a light stays on.
4. Start a stopwatch. The player has 5 seconds for each press.
5. Check buttons 1, 2, 3 and 4 in turn, over and over, many times a second.
6. Buttons "bounce": one press can flicker on and off for a few thousandths of a second. Waiting a moment and checking again ignores the flicker.
7. The matching LED stays lit while the finger is down, so the player can see what they pressed.
8. The answer goes into "pressed": the button's number, 1 to 4.
9. The stopwatch ran out with no press, so the answer is "too slow".

← The game itself

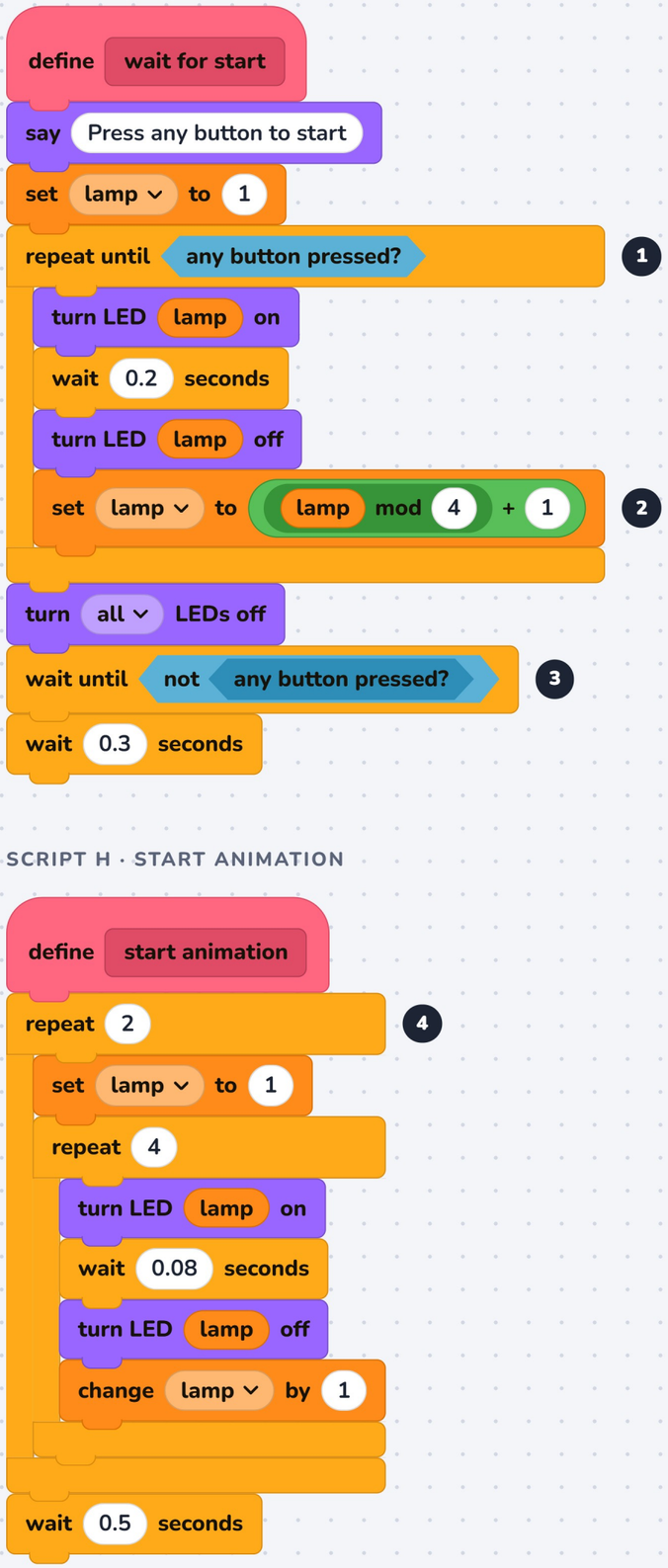
Next: lights & sounds →

SIMON SAYS · ESP32 · BOARD 4 OF 4

Lights & sounds

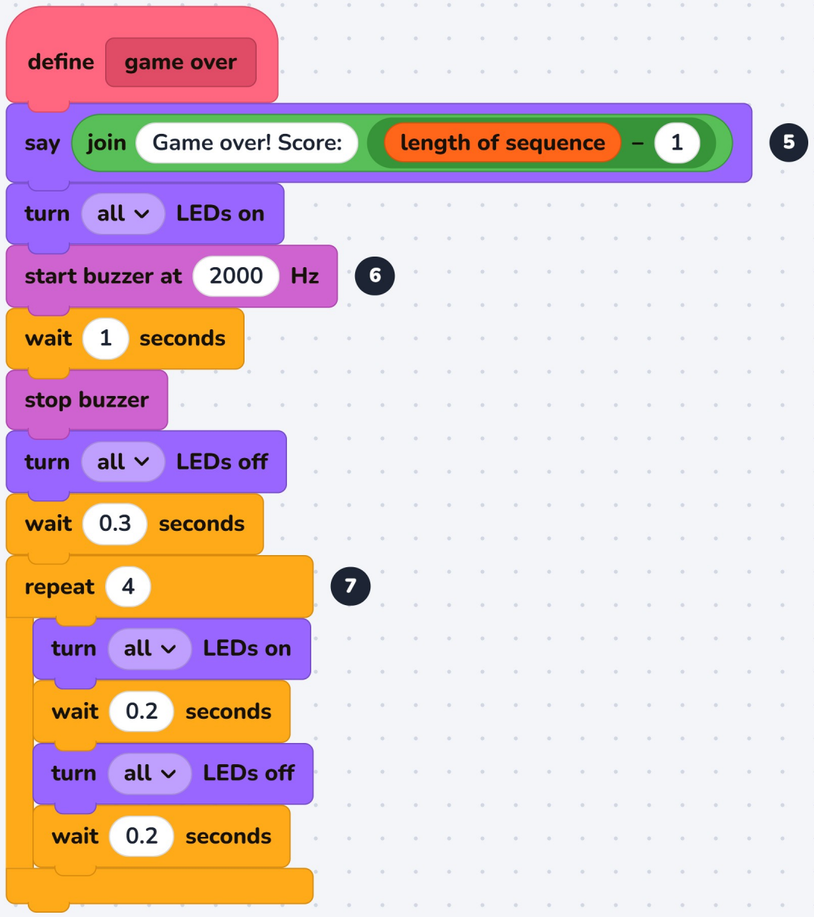
The little shows the game puts on: circling lights while it waits for a player, a quick chase before each game, a flash for every round you clear, and the buzz when you get it wrong.

SCRIPT G · WAIT FOR START

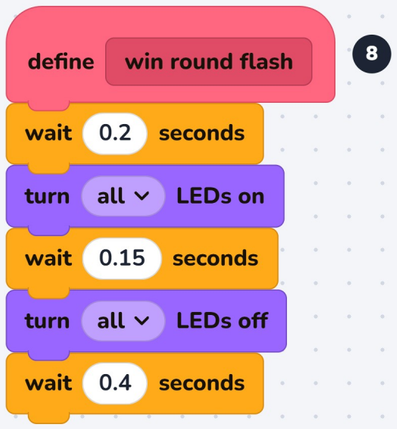


SCRIPT H · START ANIMATION

SCRIPT I · GAME OVER



SCRIPT J · WIN ROUND FLASH



WHAT'S GOING ON

1. Until someone presses a button, the lights take turns, one after another.
2. Count 1, 2, 3, 4, then back to 1, so the circle keeps going.
3. Any of the five buttons starts the game, Reset included. It waits for the finger to come off first, so that press isn't counted as the first move.
4. A fast chase along the four lights, twice, to say "get ready".
5. Your score is the number of rounds you fully cleared. The round you failed doesn't count.
6. One long, one-second buzz with every light on: you got it wrong.
7. Then all the lights blink four times.
8. A single quick blink of every light after a round you got right.

← Showing & listening

Back to the overview